

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool: Application-Based Questions

Weightage: 40%

QUESTIONS:

Total Marks: 30

1: Online Symposium Portal

A college is developing an online symposium registration portal. The following HTML structure and HTTP interaction is observed:

```
<form action="/register" method="POST">
  <input type="text" name="name">
  <input type="email" name="email">
  <button>Submit</button>
</form>
```

When a student submits the form, the browser sends an HTTP request to the server, and the server responds accordingly. **Questions:**

1. Identify appropriate **HTML5 semantic elements** missing in the structure.

The given HTML form is functional but does not follow HTML5 semantic standards. Semantic elements describe the purpose of different sections of a webpage, making the code easier to understand for developers, browsers, and search engines. They also improve accessibility for screen readers.

The missing semantic elements are:

- <header> – Displays the title or logo of the webpage.
- <nav> – Used for navigation menus.

- <main> – Contains the main content of the webpage.
 - <section> – Groups related content together.
 - <article> – Represents an independent section if required.
 - <footer> – Displays copyright information or contact details.
- Using these semantic tags improves readability, accessibility, SEO, and maintainability of the webpage.

2. Modify the structure to make it **standards-compliant**.

```
<!DOCTYPE html>
<html>
<head>
<title>Online Symposium</title>
</head>

<body>

<header>
<h1>Online Symposium Registration</h1>
</header>

<main>
<section>
<form action="/register" method="POST">

<label>Name:</label>
<input type="text" name="name" required>

<label>Email:</label>
<input type="email" name="email" required>

<button type="submit">Submit</button>

</form>
</section>
</main>

<footer>
<p>© 2026 Symposium Portal</p>
</footer>

</body>
</html>
```

3. Interpret the **HTTP request generated** during form submission.

When the student clicks the **Submit** button, the browser automatically generates an **HTTP POST request** because the form uses the POST method.

The process is:

- The user enters the name and email.
- The browser collects the form data.
- An HTTP POST request is sent to the server.
- The request URL is **/register**.
- The form data is sent inside the request body.
- The server receives the request and processes the registration.

Example HTTP Request

POST /register HTTP/1.1

Host: www.symposium.com

Content-Type: application/x-www-form-urlencoded

name=Mathusri

email=mathusri@gmail.com

Advantages of POST

- Data is not visible in the URL.
- Suitable for sensitive information.
- Supports larger amounts of data.
- More secure than GET.

4. Analyze the **HTTP response cycle** from server to browser.

After receiving the POST request, the server processes the registration and sends an HTTP response.

The response cycle is:

- **Step 1:** Browser sends HTTP POST request.
- **Step 2:** Web server receives the request.
- **Step 3:** Server validates student details.
- **Step 4:** Data is stored in the database.
- **Step 5:** Server prepares an HTTP response.
- **Step 6:** Browser receives the response.
- **Step 7:** Confirmation page is displayed.

Example HTTP Response

HTTP/1.1 200 OK

Content-Type: text/html

Registration Successful

Possible response codes include:

- **200 OK** – Registration successful.
- **400 Bad Request** – Invalid input.
- **404 Not Found** – Resource not found.
- **500 Internal Server Error** – Server problem.

The browser interprets the response and displays the appropriate webpage to the user.

5. Suggest improvements for **usability and accessibility**.

The portal can be improved by implementing the following features:

Usability Improvements

- Add placeholders inside input fields.
- Display clear error messages for invalid input.
- Use responsive design for mobile devices.
- Add a Reset button if necessary.
- Display a success confirmation after submission.

Accessibility Improvements

- Use <label> for every input field.
- Use semantic HTML5 elements.
- Ensure keyboard navigation support.
- Maintain sufficient color contrast.
- Add ARIA attributes where necessary.
- Support screen readers.
- Use meaningful button names.

Benefits

- Better user experience.
- Improved accessibility.
- Higher search engine ranking.
- Easier maintenance.
- Compliance with HTML5 and W3C standards.

Conclusion

The Online Symposium Portal can be significantly improved by using HTML5 semantic elements, implementing a standards-compliant structure, understanding the HTTP request-response cycle, and enhancing usability and accessibility. These improvements make the application more secure, user-friendly, maintainable, and compatible across modern web browsers.

2: Cross-Browser Compatibility Issue

A company website shows inconsistent layout across browsers. The following HTML snippet is used:

```
<html>
  <head>
    <title>Company</title>
  </head>
  <body>
    <div>
      <h1>Welcome
      <p>About us
    </div>
  </body> </html>
```

Questions:

1. Identify improper nesting and missing closing tags.

The given HTML code contains several syntax errors that make it invalid according to HTML5 standards.

Given Code

```
HTML
<html>
<head>
<title>Company</title>
</head>
<body>

<div>

<h1>Welcome

<p>About us

</div>
</body>
</html>
```

Errors Identified

1. Missing <!DOCTYPE html> declaration.
2. <h1> tag is not closed using </h1>.
3. <p> tag is not closed using </p>.
4. Semantic HTML5 elements are not used.
5. The content is placed inside a generic <div> instead of meaningful tags.
6. Improper nesting may lead to unpredictable browser behavior.

Explanation

Improper nesting means HTML elements are not opened and closed correctly. Browsers may try to fix these mistakes automatically, but different browsers may interpret the page differently. Therefore, all HTML tags should be properly nested and closed according to W3C standards.

2. Analyze how different browsers may interpret this structure.

Different web browsers have different rendering engines.

- **Google Chrome** uses the Blink engine.
- **Mozilla Firefox** uses the Gecko engine.
- **Microsoft Edge** uses the Blink engine.
- **Safari** uses the WebKit engine.

Since the HTML code is not properly structured, each browser attempts to correct the errors automatically.

Possible Effects

- Text alignment may change.
- Font sizes may vary.
- Headings may appear differently.
- Paragraph spacing may differ.
- CSS styles may not work consistently.
- JavaScript may behave unexpectedly.

Example

Chrome may automatically close the <h1> tag before the paragraph, while another browser may treat the paragraph as part of the heading. This causes inconsistent webpage layouts.

Conclusion

Proper HTML coding ensures that every browser displays the webpage in the same way.

3. Identify missing semantic elements (e.g., header, section).

The webpage uses only a <div> element, which does not clearly describe the purpose of the content.

The following semantic elements should be used:

- <header> – Displays the company title.
- <main> – Contains the main webpage content.
- <section> – Groups related information.
- <article> – Represents independent content if needed.
- <footer> – Displays copyright information.

Benefits of Semantic Elements

- Improves readability.
- Enhances accessibility.
- Better search engine optimization (SEO).
- Easier maintenance.
- Makes the webpage more structured.

4. Provide a corrected W3C-compliant HTML structure.

HTML

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
<title>Company</title>
```

```
</head>
```

```
<body>
```

```
<header>
```

```
<h1>Welcome</h1>
```

```
</header>
```

```
<main>
```

```
<section>
```

```
<p>
```

```
About us
```

```
</p>
```

```
</section>
```

```
</main>
```

```
<footer>
```

```
<p>
```

```
© 2026 Company. All Rights Reserved.
```

```
</p>
```

```
</footer>
```

```
</body>
```

```
</html>
```

Explanation

The corrected HTML structure includes:

- HTML5 doctype declaration.
- Language attribute (lang="en").
- Character encoding (UTF-8).
- Responsive viewport.
- Semantic elements.
- Proper opening and closing tags

5. Suggest techniques to ensure cross-browser compatibility.

To ensure that a webpage works consistently across different browsers, developers should follow these techniques:

1. Validate HTML Code

Use the W3C HTML Validator to identify and correct coding errors.

2. Follow HTML5 Standards

Use valid HTML5 tags and avoid deprecated elements.

3. Use Semantic Elements

Use <header>, <main>, <section>, <footer>, and other semantic tags to organize content.

4. Test in Multiple Browsers

Check the webpage in:

- Google Chrome
- Mozilla Firefox
- Microsoft Edge
- Safari

5. Use Responsive Design

Use CSS media queries and flexible layouts to support different screen sizes.

6. Normalize CSS

Use CSS Reset or Normalize.css to reduce browser default styling differences.

7. Keep Software Updated

Use the latest browser versions to ensure compatibility with modern web standards.

8. Avoid Browser-Specific Code

Write standard HTML, CSS, and JavaScript instead of relying on browser-specific features.

Conclusion

Cross-browser compatibility ensures that a website provides a consistent user experience across all modern web browsers. By writing valid HTML5 code, using semantic elements, properly closing tags, validating the webpage, and testing across multiple browsers, developers can create reliable, accessible, and standards-compliant web applications that work efficiently for all users.

3: Form Submission Failure

A web application fails to send form data to the server. The following configuration is observed:

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Questions:

1. Identify the issues in the form configuration

The given form contains the following problems:

HTML

```
<form action="/submit" method="GET">
  <input type="text" name="username">
  <input type="password" name="password">
  <button type="button">Login</button>
</form>
```

Problems identified

- The button uses type="button" instead of type="submit".
- The form uses the GET method for sending login details.
- The password may appear in the browser URL.
- The input fields do not have <label> elements.
- The fields do not use the required attribute.
- There is no secure HTTPS connection mentioned.
- There is no client-side or server-side validation.

The most important issue is the button type. A normal button does not automatically submit the form.

2. Analyze why the HTTP request is not triggered

The HTTP request is not triggered because the login button is written as:

HTML

```
<button type="button">Login</button>
```

A button with type="button" only behaves like a normal clickable button. It does not submit the form unless JavaScript is added to handle the click event.

To submit the form automatically, the button must be:

HTML

```
<button type="submit">Login</button>
```

When the user clicks a submit button, the browser performs the following steps:

- Checks the form fields.
- Collects the username and password.
- Creates an HTTP request.
- Sends the request to /submit.
- Waits for the server response.

3. Interpret the role of the GET method in this scenario

The GET method is mainly used to retrieve information from a server.

For example:

`https://example.com/search?keyword=html`

When GET is used, form data is usually appended to the URL as query parameters.

In the given login form, the URL may appear as:

`/submit?username=student&password=12345`

This is unsafe because:

- The password becomes visible in the address bar.
- The URL may be stored in browser history.
- It may be saved in server logs.
- It can be copied or shared accidentally.
- GET requests may be cached.

Therefore, GET is suitable for search forms and data retrieval, but it should not be used for transmitting passwords or other sensitive details.

4. Propose a corrected implementation

HTML

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>User Login</title>
```

```
</head>
```

```
<body>
```

```
<header>
```

```
  <h1>Login Page</h1>
```

```
</header>
```

```
<main>
```

```
<section>
```

```
<form action="/submit" method="POST">
```

```
  <label for="username">Username:</label><br>
```

```
  <input
```

```
    type="text"
```

```
    id="username"
```

```
    name="username"
```

```
    required
```

```

        autocomplete="username">
    <br><br>

    <label for="password">Password:</label><br>
    <input
        type="password"
        id="password"
        name="password"
        required
        autocomplete="current-password">
    <br><br>

    <button type="submit">Login</button>

</form>

</section>

</main>

<footer>
    <p>Secure Login Portal</p>
</footer>

</body>
</html>

```

Explanation of the corrected code

- method="POST" sends data in the request body.
- type="submit" triggers form submission.
- <label> elements improve accessibility.
- required prevents empty submission.
- id connects each label with its input field.
- autocomplete helps browsers handle login information properly.
- Semantic tags improve structure and readability.

5. Suggest improvements for secure data transmission

Use HTTPS

The website must use HTTPS instead of HTTP. HTTPS encrypts data transmitted between the browser and server.

Example:

<https://example.com/submit>

Use the POST method

POST keeps the username and password out of the URL. However, POST alone does not encrypt the data, so HTTPS is still required.

Validate input on the server

The server should verify:

- Username format
- Password length
- Empty values
- Invalid characters
- Malicious input

Client-side validation alone is not sufficient because it can be bypassed.

Hash passwords

Passwords should never be stored as plain text. They should be hashed using secure password-hashing methods such as bcrypt, Argon2, or PBKDF2.

Prevent SQL injection

Use prepared statements or parameterized queries instead of directly inserting user input into SQL commands.

Use secure sessions

After successful login, the server should create a secure session. Session cookies should use:

- Secure
- HttpOnly

Add CSRF protection

A CSRF token should be included in forms to prevent unauthorized requests from other websites.

Limit repeated login attempts

The system should temporarily block or slow down repeated failed login attempts to reduce brute-force attacks.

Display safe error messages

Avoid messages such as:

Username exists, but password is incorrect.

Instead, display:

Invalid username or password.

This prevents attackers from identifying valid usernames.

Example HTTP POST Request

After correction, the browser may send:

POST /submit HTTP/1.1

Host: www.example.com

Content-Type: application/x-www-form-urlencoded

username=student&password=12345

The data is placed in the request body instead of the URL.

Example HTTP Response

For successful login:

HTTP/1.1 200 OK

Content-Type: text/html

Login Successful

For invalid login:

HTTP/1.1 401 Unauthorized

Content-Type: text/html

Invalid username or password

Conclusion

The form fails because the login button is a normal button and does not submit the form. The use of GET is also unsuitable because login credentials may become visible in the URL. Changing the button to type="submit", using the POST method, enabling HTTPS, validating input, hashing passwords, and using secure session management will make the login form functional and more secure.

Code of Conduct:

I Mathusri P-192311363 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Mathusri P

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand